

Eperiment 7: Depth First Search (DFS)

Aim

To implement Depth First Search (DFS) for traversing a graph and demonstrate depth-wise state-space search.

Algorithm

1. Start
2. Initialize the graph and starting vertex.
3. Mark the starting vertex as visited.
3. Mark the starting vertex as visited.
4. Display the current vertex.
5. Recursively visit every unvisited adjacent vertex.
6. Backtrack when no unvisited adjacent vertex remains.
7. Stop when all reachable vertices are visited.
8. Stop.

Flowchart:

Start



Initialize



Process / Search



Goal / Condition Reached?

n n

Yes No



Print Result Repeat



Stop

Objective

- Understand depth-first state-space search.
- Implement DFS using recursion.

- Traverse all reachable vertices from a source vertex.

Program

```
def dfs(graph, vertex, visited=None):  
    if visited is None:  
        visited = set()  
        visited.add(vertex)  
        print(vertex, end=" ")  
        for neighbour in graph.get(vertex, []):  
            if neighbour not in visited:  
                dfs(graph, neighbour, visited)  
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': [],  
    'F': []  
}  
print("DFS Traversal:")  
dfs(graph, 'A')
```

Output

DFS Traversal:

A B D E C F

Result

The program was executed successfully and produced the expected result.

Conclusion

The experiment successfully demonstrated the corresponding Artificial Intelligence technique and its application